



งานปฏิบัติการกลุ่ม StackQueue

ผู้จัดทำ

นายสรสิช มาศสิน 68122250006

นาย เอกราช บุตรรักษ์ 68122250007

นาย ภาพตะวัน สายแก้ว 68122250012

นางสาว ณัฐชา ขำทองทับ 68122250037

เสนอ

ผศ.ดร. กัลยณัฐ กุหลาบเพชรทอง

รายงานนี้เป็นส่วนหนึ่งของวิชา CSD2103

การออกแบบและวิเคราะห์ขั้นตอนวิธี

สาขาวิชาวิทยาการคอมพิวเตอร์และนวัตกรรมข้อมูล

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏสวนสุนันทา

ภาคเรียนที่ 1 ปีการศึกษา 2569

บทที่ 1 บทนำ

ที่มาและความสำคัญ

ในปัจจุบันลานจอดรถหลายแห่งมีพื้นที่จำกัดและมีทางเข้าออกเพียงช่องทางเดียว ทำให้รถยนต์ที่เข้ามาจอดก่อนจะอยู่ลึกเข้าไปภายในลานจอด ส่วนรถยนต์ที่เข้ามาภายหลังจะอยู่ใกล้ทางออกมากกว่า เมื่อต้องการนำรถคันที่อยู่ด้านในออกจากลานจอด จำเป็นต้องเคลื่อนย้ายรถที่จอดขวางอยู่ด้านหน้าออกชั่วคราวก่อน และเมื่อรถเป้าหมายออกจากลานจอดแล้ว จะต้องนำรถที่ถูกย้ายออกชั่วคราวกลับเข้าที่เดิมเพื่อรักษาลำดับการจอดรถ

ลักษณะการทำงานดังกล่าวสอดคล้องกับหลักการของโครงสร้างข้อมูลแบบสแตค (Stack) ซึ่งทำงานตามหลัก Last In First Out (LIFO) หรือข้อมูลที่เข้าสู่โครงสร้างข้อมูลเป็นลำดับสุดท้ายจะถูกนำออกก่อน จึงเหมาะสมสำหรับใช้ในการจำลองและแก้ปัญหาการจัดการลานจอดรถที่มีทางเข้าออกเพียงด้านเดียว

โครงการนี้จัดทำขึ้นเพื่อศึกษา ออกแบบ และวิเคราะห์อัลกอริทึมที่ใช้โครงสร้างข้อมูลแบบ Stack ในการค้นหาและนำรถเป้าหมายออกจากลานจอดรถ โดยออกแบบอัลกอริทึมอย่างน้อย 2 วิธี ได้แก่ การทำงานแบบวนซ้ำ (Iterative Method) และการทำงานแบบเรียกซ้ำ (Recursive Method) พร้อมทั้งเปรียบเทียบประสิทธิภาพด้านเวลาและการใช้หน่วยความจำ เพื่อเลือกวิธีที่เหมาะสมสำหรับการนำไปประยุกต์ใช้งานจริง

Case Story

ลานจอดรถแห่งหนึ่งมีทางเข้าและทางออกเพียงด้านเดียว ทำให้รถที่เข้ามาจอดก่อนจะอยู่ลึกเข้าไปด้านใน ส่วนรถที่เข้ามาทีหลังจะอยู่ใกล้ทางออกมากกว่า

เมื่อต้องการนำรถคันใดคันหนึ่งออกจากลานจอด รถที่ขวางอยู่ด้านหน้าจะต้องถูกนำออกชั่วคราวก่อน เมื่อพบรถเป้าหมายและนำออกแล้ว รถที่ถูกย้ายออกชั่วคราวจะต้องถูกนำกลับเข้ามายังลานจอดในลำดับเดิม เพื่อรักษาสภาพการจอดรถให้เหมือนก่อนการนำรถเป้าหมายออก

เนื่องจากลักษณะการเข้าออกแบบ LIFO (Last In First Out) ปัญหานี้จึงเหมาะสมกับการใช้โครงสร้างข้อมูล Stack

วัตถุประสงค์

1. เพื่อศึกษาแนวคิด หลักการทำงาน และคุณสมบัติของโครงสร้างข้อมูลแบบสแตค (Stack) รวมถึงการประยุกต์ใช้ในการแก้ปัญหาการจัดการลานจอดรถที่มีทางเข้าและทางออกเพียงด้านเดียว
2. เพื่อออกแบบและพัฒนาอัลกอริทึมสำหรับค้นหาและนำรถยนต์เป้าหมายออกจากลานจอดรถ โดยใช้โครงสร้างข้อมูลแบบ Stack เป็นโครงสร้างข้อมูลหลักในการจัดเก็บและจัดการข้อมูล

3. เพื่อพัฒนาอัลกอริทึมอย่างน้อย 2 วิธี ได้แก่ วิธีการแบบวนซ้ำ (Iterative Method) และวิธีการแบบเรียกซ้ำ (Recursive Method) พร้อมศึกษาหลักการทำงาน ข้อดี และข้อจำกัดของแต่ละวิธี
4. เพื่อวิเคราะห์ความถูกต้องของอัลกอริทึมที่ออกแบบขึ้น โดยใช้หลักการพิสูจน์ความถูกต้อง เช่น Loop Invariant หรือวิธีการอื่นที่เหมาะสม เพื่อยืนยันว่าอัลกอริทึมสามารถนำรถเป้าหมายออกจากลานจอดได้อย่างถูกต้องและรักษาลำดับของรถที่เหลืออยู่ได้
5. เพื่อวิเคราะห์ประสิทธิภาพของอัลกอริทึมในด้านเวลาในการทำงาน (Time Complexity) และการใช้หน่วยความจำ (Space Complexity) ทั้งในกรณี Best Case, Average Case และ Worst Case
6. เพื่อพัฒนาโปรแกรมภาษา Java ที่สามารถทำงานตามอัลกอริทึมที่ออกแบบไว้ได้จริง มีระบบตรวจสอบข้อมูลนำเข้า การแสดงสถานะของ Stack การวัดเวลาในการทำงาน และการนับจำนวนการดำเนินการที่เกิดขึ้นภายในโปรแกรม
7. เพื่อทดสอบประสิทธิภาพของอัลกอริทึมทั้งสองวิธีด้วยชุดข้อมูลหลายขนาด พร้อมเปรียบเทียบผลการทำงานในด้านเวลา จำนวน Operation และการใช้หน่วยความจำ เพื่อเลือกแนวทางที่เหมาะสมที่สุดสำหรับการนำไปประยุกต์ใช้งานจริง
8. เพื่อฝึกทักษะการวิเคราะห์ปัญหา การออกแบบอัลกอริทึม การพัฒนาโปรแกรม การทำงานเป็นทีม และการจัดทำเอกสารรายงานทางวิชาการตามกระบวนการพัฒนาซอฟต์แวร์อย่างเป็นระบบ

ขอบเขตงาน

โครงการนี้มุ่งเน้นการศึกษา ออกแบบ พัฒนา และวิเคราะห์อัลกอริทึมสำหรับการจัดการลานจอดรถที่มีทางเข้าและทางออกเพียงด้านเดียว โดยใช้โครงสร้างข้อมูลแบบ Stack เป็นโครงสร้างข้อมูลหลักในการจัดเก็บและจัดการข้อมูลรถยนต์ภายในลานจอดรถ

ขอบเขตของโครงการมีรายละเอียดดังนี้

1. ศึกษาและวิเคราะห์ปัญหาการนำรถยนต์เป้าหมายออกจากลานจอดรถ โดยคำนึงถึงเงื่อนไขที่รถยนต์บางคันอาจจอดขวางอยู่ด้านหน้า ทำให้ต้องมีการเคลื่อนย้ายรถออกชั่วคราวก่อน
 2. ออกแบบอัลกอริทึมสำหรับการค้นหาและนำรถเป้าหมายออกจากลานจอดรถจำนวน 2 วิธี ได้แก่
 - Algorithm A: Iterative Method
 - Algorithm B: Recursive Method
 3. ใช้โครงสร้างข้อมูลแบบ Stack เป็นโครงสร้างข้อมูลหลักในการจัดเก็บข้อมูลรถยนต์ และใช้สำหรับการดำเนินการต่าง ๆ ภายในอัลกอริทึม
-
1. พัฒนาโปรแกรมด้วยภาษา Java โดยให้โปรแกรมสามารถ

- เพิ่มข้อมูลรถยนต์เข้าสู่ Stack
 - แสดงข้อมูลรถยนต์ภายใน Stack
 - ค้นหารถยนต์เป้าหมาย
 - นำรถยนต์เป้าหมายออกจากลานจอด
 - แสดงสถานะของ Stack ก่อนและหลังการดำเนินการ
 - แสดงจำนวน Operation ที่เกิดขึ้น
 - วัดเวลาในการทำงานของอัลกอริทึม
2. วิเคราะห์ความถูกต้องของอัลกอริทึมโดยใช้หลักการพิสูจน์ความถูกต้อง (Algorithm Correctness) เพื่อยืนยันว่าอัลกอริทึมสามารถทำงานได้อย่างถูกต้องตามวัตถุประสงค์
 3. วิเคราะห์ประสิทธิภาพของอัลกอริทึมในด้านเวลาในการทำงาน (Time Complexity) และการใช้หน่วยความจำ (Space Complexity) ในกรณี Best Case, Average Case และ Worst Case
 4. ทดสอบประสิทธิภาพของอัลกอริทึมด้วยชุดข้อมูลหลายขนาด ได้แก่ 100, 1,000, 10,000 และ 50,000 รายการ โดยทดสอบอย่างน้อย 5 รอบต่อขนาดข้อมูล และใช้ค่าเฉลี่ยในการวิเคราะห์ผล
 5. จัดทำ Test Cases อย่างน้อย 8 กรณี ได้แก่ Normal Case, Best Case, Worst Case, Empty Input, Invalid Input, Target Not Found, Boundary Case และ System-specific Case
 6. เปรียบเทียบประสิทธิภาพของอัลกอริทึมทั้งสองวิธีจากผลการทดลองจริง พร้อมจัดทำตารางและกราฟแสดงผลเพื่อใช้ในการวิเคราะห์และสรุปผล
 7. จัดทำเอกสารประกอบโครงการ ได้แก่ รายงานฉบับสมบูรณ์ สไลด์นำเสนอ GitHub Repository Flowchart Class Diagram ผลการทดลอง และบันทึกการใช้ Generative AI ตามข้อกำหนดของรายวิชา

บทที่ 2 การวิเคราะห์ปัญหา

2.1 Input

ข้อมูลรถแต่ละคันประกอบด้วย

- License Plate
- Owner Name
- Entry Time
- Vehicle Type

และ

- License Plate ของรถเป้าหมาย

ตัวอย่าง

- A111
- B222
- C333
- D444
- Target = B222

2.2 Output

- ผลการค้นหารถเป้าหมาย
- จำนวนรถที่ถูกย้าย
- สถานะ Stack หลังดำเนินการ
- จำนวน Push
- จำนวน Pop
- จำนวน Comparisons
- เวลาที่ใช้ในการทำงาน
- ตัวอย่าง

Target Found : B222

Cars Moved : 2

Push : 2

Pop : 3

Execution Time : 1200 ns

2.3 Constraints

1. ใช้ Stack เป็นโครงสร้างข้อมูลหลัก
2. ต้องออกแบบอัลกอริทึม 2 วิธี
3. ต้องรักษาลำดับรถคงเดิมหลังนำรถเป้าหมายออก
4. ข้อมูลทะเบียนรถต้องไม่ซ้ำกัน
5. ต้องสามารถจัดการกรณี Stack วางได้

2.4 Assumptions

1. ข้อมูลที่รับเข้าถูกต้อง
2. ทะเบียนรถไม่ซ้ำกัน
3. Stack มีขนาดเพียงพอ
4. รถเป้าหมายระบุด้วย License Plate

2.5 Edge Cases

Case 1 : Stack ว่าง

[]

ผลลัพธ์

Target Not Found

Case 2 : รถอยู่ด้านบนสุด

TOP

B

A

S

BOTTOM

สามารถนำออกได้ทันที

Case 3 : รถอยู่ล่างสุด

TOP

D

C

B

A

BOTTOM

ต้องย้ายรถด้านบนทั้งหมดก่อน

Case 4 : ไม่พบรถเป้าหมาย

Target = X

ผลลัพธ์

Not Found

Case 5 : มีรถเพียง 1 คัน

TOP

A

BOTTOM

บทที่ 3 การออกแบบอัลกอริทึม

Algorithm A : Iterative Method

แนวคิด

ใช้งาน Stack จำนวน 2 ตัว

- parkingStack
- temporaryStack

เมื่อตรวจพบรถที่ไม่ใช่เป้าหมาย จะนำไปเก็บไว้ใน temporaryStack ชั่วคราว

หลังจากนำรถเป้าหมายออกแล้ว จะนำรถทั้งหมดกลับเข้าสู่ parkingStack เพื่อรักษาลำดับเดิม

Pseudocode Algorithm A : Iterative Method

Algorithm A: Iterative Stack Search

Input: target license plate

Output: ParkingRecord

1. Create ParkingRecord
2. While parkingStack is not empty
 - 2.1 Pop a vehicle from parkingStack
 - 2.2 Increase Pop Count
 - 2.3 Increase Comparison Count
 - 2.4 If vehicle license plate equals target

```

        Set found = true

        Stop searching

    Else

        Push vehicle into temporaryStack

        Increase Push Count

        Increase Cars Moved

3. While temporaryStack is not empty

    3.1 Pop vehicle from temporaryStack

    3.2 Push vehicle back into parkingStack

    3.3 Increase Push Count

4. Return ParkingRecord

```

ข้อดี

- เข้าใจง่าย
- ไม่เกิด Stack Overflow
- ควบคุมการทำงานได้ง่าย

ข้อจำกัด

- ใช้ Temporary Stack เพิ่ม

Algorithm B : Recursive Method

แนวคิด

ใช้ Recursive Call แทน Temporary Stack

โดยให้ Call Stack ของระบบทำหน้าที่เก็บข้อมูลชั่วคราว

Pseudocode Algorithm B : Recursive Method

Algorithm B: Recursive Stack Search

Input: target license plate

Output: ParkingRecord

1. Create ParkingRecord
2. Call RecursiveSearch(target, ParkingRecord)

RecursiveSearch(target, record)

3. If parkingStack is empty

Return false

4. Increase Loop Count

5. Pop a vehicle from parkingStack

6. Increase Pop Count

7. Increase Comparison Count

8. If vehicle license plate equals target

Set found = true

Return true

9. Call RecursiveSearch(target, record)

10. Push the removed vehicle back into parkingStack

11. Increase Push Count

12. If target was not found

Increase Cars Moved

13. Return search result

ข้อดี

- โค้ดสั้น
- เขียนง่าย
- แสดงแนวคิด Recursion ชัดเจน

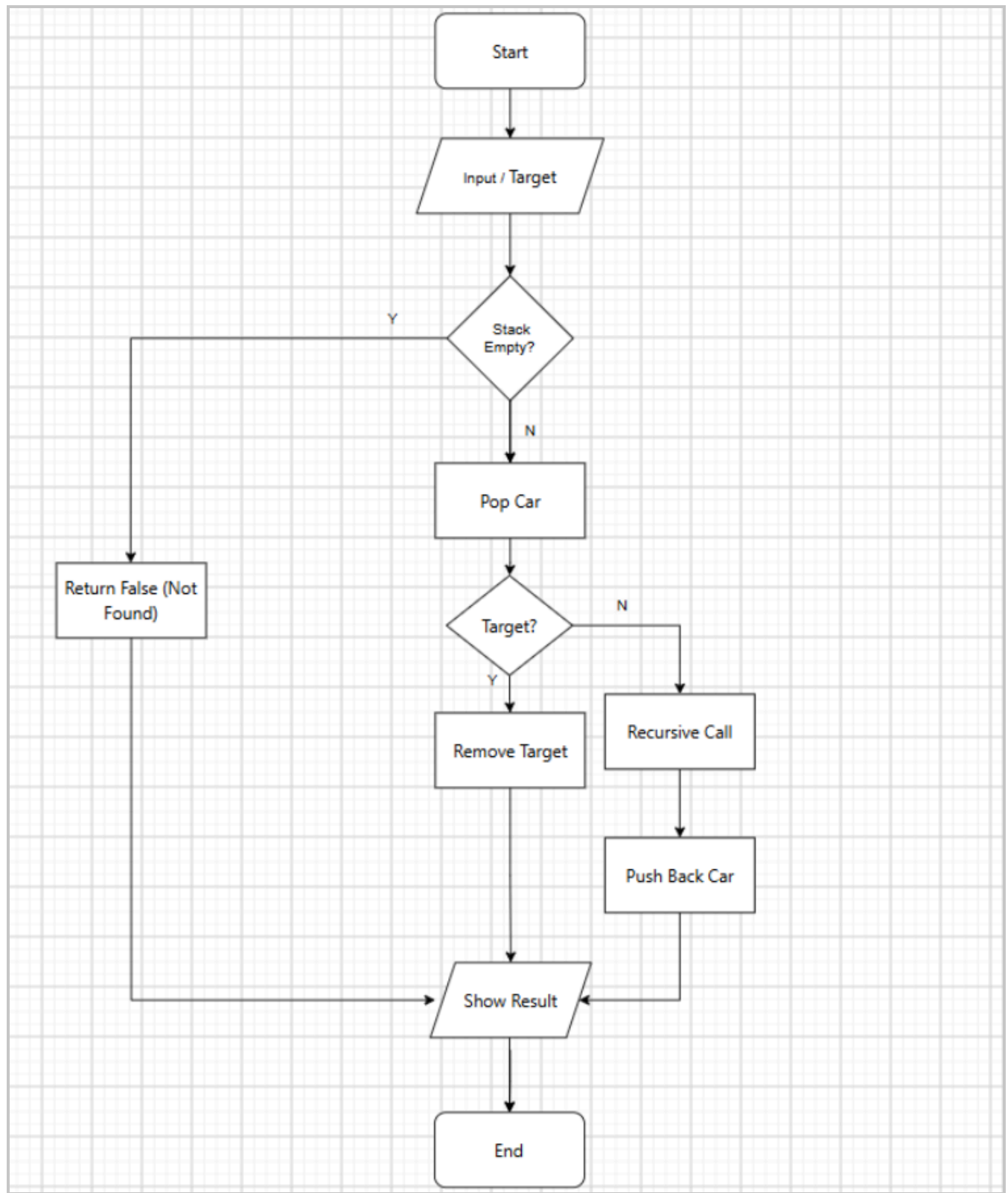
ข้อจำกัด

- ใช้หน่วยความจำเพิ่มจาก Call Stack
- เสี่ยง Stack Overflow

Flowchart Algorithm A



Flowchart Algorithm B



Step-by-Step (Normal Case)

Initial Stack

TOP

D

C

B

A

BOTTOM

Target = B

Step 1

Pop D

Temp:

D

Step 2

Pop C

Temp:

C

D

Step 3

พบ B

นำ B ออก

Step 4

Push C กลับ

Step 5

Push D กลับ

Final Stack

TOP

D

C

A

BOTTOM

Step-by-Step (Worst Case)

Target = A

Initial

TOP

D

C

B

A

BOTTOM

Step 1

Pop D

Step 2

Pop C

Step 3

Pop B

Step 4

พบ A

Step 5

คืน B

Step 6

คืน C

Step 7

คืน D

Final

TOP

D

C

B

BOTTOM

บทที่ 4 การอธิบายความถูกต้อง

วิธีที่เลือก : Loop Invariant

Loop Invariant

ก่อนเริ่มการทำงานในแต่ละรอบของลูป

รถทุกคันที่ถูกนำออกจาก parkingStack และไม่ใช่รถเป้าหมาย จะถูกเก็บไว้ใน temporaryStack โดยไม่มีข้อมูลสูญหาย และรถที่ยังอยู่ใน parkingStack ยังคงลำดับเดิม

Initialization

ก่อนเริ่มลูป

temporaryStack ว่าง

ดังนั้น Invariant เป็นจริง

Maintenance

เมื่อ Pop รถออกจาก parkingStack

หากไม่ใช่รถเป้าหมาย

temporaryStack.push(car)

รถถูกเก็บครบถ้วน ไม่มีข้อมูลสูญหาย

ดังนั้น Invariant ยังคงเป็นจริง

Termination

ลูปจะจบเมื่อ

พบรถเป้าหมาย

หรือ

Stack ว่าง

หลังจากนั้นนำรถทั้งหมดใน temporaryStack กลับไปยัง parkingStack

ลำดับเดิมจึงถูกรักษาไว้

Correctness

อัลกอริทึมถูกต้องเนื่องจาก

1. รถทุกคันถูกตรวจสอบอย่างน้อย 1 ครั้ง
2. ไม่มีข้อมูลสูญหาย
3. รถเป้าหมายถูกนำออกเมื่อพบ
4. รถที่เหลือกลับสู่ลำดับเดิม
5. ลูบสิ้นสุดแน่นอนเพราะจำนวนข้อมูลลดลงทุกครั้งที่ Pop

บทที่ 5 การวิเคราะห์ประสิทธิภาพ

Algorithm A : Iterative

Best Case

รถอยู่บนสุด

$O(1)$

Average Case

รถอยู่ตำแหน่งกลาง

$O(n)$

Worst Case

รถอยู่ล่างสุดหรือไม่พบ

$O(n)$

Auxiliary Space

$O(n)$

ใช้ temporaryStack

Total Space

$O(n)$

Algorithm B : Recursive

Best Case

$O(1)$

Average Case

$O(n)$

Worst Case

$O(n)$

Auxiliary Space

$O(n)$

จาก Call Stack

Total Space

$O(n)$

ตารางเปรียบเทียบ

Algorithm	Best Case	Average Case	Worst Case	Auxiliary Space
Iteration	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Recursion	$O(1)$	$O(n)$	$O(n)$	$O(n)$

สรุปการเปรียบเทียบ

- Iteration เหมาะกับข้อมูลขนาดใหญ่
- Recursion เขียนง่ายและสั้นกว่า
- ทั้งสองอัลกอริทึมมี Time Complexity เท่ากันคือ $O(n)$
- Recursion มีความเสี่ยง Stack Overflow มากกว่า
- หากต้องใช้งานจริงในระบบงานจริง ควรเลือก Iteration เนื่องจากมีความเสถียรมากกว่าและควบคุมหน่วยความจำได้ดีกว่า

Class Diagram

โปรแกรมระบบจัดการลานจอดรถพัฒนาด้วยภาษา Java โดยแบ่งการทำงานออกเป็น 4 คลาสหลัก ได้แก่ **Main**, **Vehicle**, **ParkingRecord** และ **StackManager**

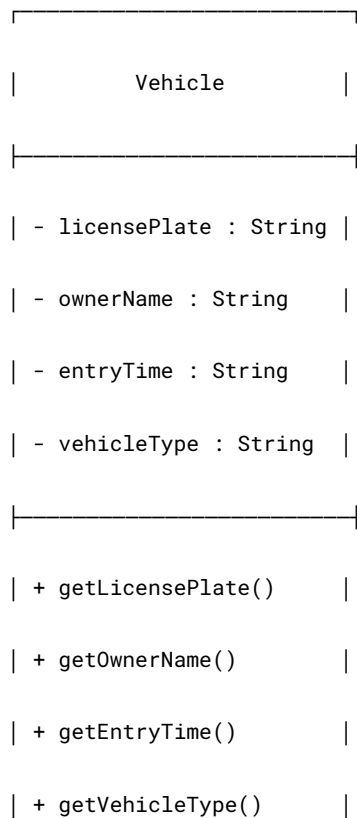
Main ทำหน้าที่ควบคุมการทำงานและแสดงเมนู

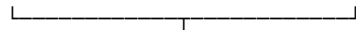
Vehicle ทำหน้าที่เก็บข้อมูลรถ

ParkingRecord ทำหน้าที่เก็บสถิติการทำงาน

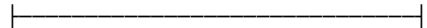
StackManager ทำหน้าที่จัดการ **parkingStack** และ

Class Diagram



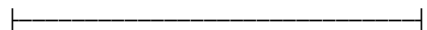


| StackManager |



| - parkingStack |

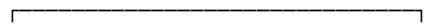
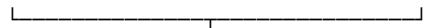
| - temporaryStack |



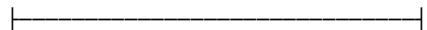
| + addVehicle() |

| + removeVehicle() |

| + displayStack() |



| ParkingRecord |

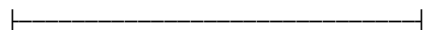


| - pushCount : int |

| - popCount : int |

| - comparisonCount : int |

| - carsMoved : int |



| + incrementPush() |


```

| + incrementPop()          |
| + incrementComparison()   |
|_____|

```

```

|_____|
|           Main           |
|_____|
| + main()                 |
| + addVehicle()           |
| + removeVehicle()        |
| + displayStack()         |
|_____|

```

โครงสร้างคลาส

Vehicle Class

Vehicle เป็นคลาสที่ใช้สำหรับเก็บข้อมูลของรถแต่ละคัน ประกอบด้วย

licensePlate — เลขทะเบียนรถ

ownerName — ชื่อเจ้าของรถ

entryTime — เวลาเข้าลานจอด

vehicleType — ประเภทรถ

ตัวอย่างข้อมูล:

License Plate : B222

Owner Name : BBB

Entry Time : 11:00

Vehicle Type : Car

ParkingRecord Class

ParkingRecord เป็นคลาสที่ใช้เก็บข้อมูลสถิติจากการทำงานของโปรแกรม ได้แก่

จำนวน Push

จำนวน Pop

จำนวน Comparison

จำนวนรถที่ถูกย้าย

สถานะการพบรถเป้าหมาย

เวลาที่ใช้ในการทำงาน

ข้อมูลเหล่านี้ใช้สำหรับตรวจสอบผลการทำงานและวิเคราะห์ประสิทธิภาพของ Algorithm

StackManager Class

StackManager เป็นคลาสที่ทำหน้าที่จัดการข้อมูลรถภายใน Stack โดยใช้ Stack 2 ตัว ได้แก่

parkingStack

ใช้เก็บรถที่อยู่ภายในลานจอดรถ

temporaryStack

ใช้เก็บรถที่ถูกนำออกชั่วคราวระหว่างค้นหารถเป้าหมาย

การจัดเก็บรถใช้หลัก LIFO (Last In First Out)

Main Class

Main เป็นคลาสหลักของโปรแกรม ทำหน้าที่รับข้อมูลจากผู้ใช้และควบคุมการทำงานผ่านเมนู ได้แก่

1. Add Vehicle
2. Remove Vehicle

3. Display Stack

4. Exit

เมธอดสำคัญ

เมธอดสำคัญที่ใช้ในโปรแกรม ได้แก่

<code>addVehicle()</code>	เพิ่มรถเข้าสู่ Stack
<code>removeVehicle()</code>	ค้นหาและนำรถเป้าหมายออก
<code>displayStack()</code>	แสดงสถานะของ Stack
<code>push()</code>	เพิ่มข้อมูลเข้าสู่ Stack
<code>pop()</code>	นำข้อมูลออกจาก Stack
<code>isEmpty()</code>	ตรวจสอบว่า Stack ว่างหรือไม่
<code>getLicensePlate()</code>	อ่านเลขทะเบียนรถ
<code>incrementPush()</code>	เพิ่มจำนวน Push
<code>incrementPop()</code>	เพิ่มจำนวน Pop
<code>incrementComparison()</code>	เพิ่มจำนวน Comparison

เมธอดที่สำคัญที่สุดของ Algorithm A คือ `removeVehicle()` ซึ่งใช้การวนลูปในการนำรถออกจาก `parkingStack` และใช้ `temporaryStack` เก็บรถที่ขวางรถเป้าหมาย

Input Validation

โปรแกรมมีการตรวจสอบข้อมูลที่ใช้ป้อนก่อนนำไปใช้งาน เพื่อป้องกันข้อมูลที่ไม่ถูกต้อง

ตรวจสอบข้อมูลว่าง

ข้อมูลสำคัญ เช่น License Plate และ Owner Name ต้องไม่เป็นค่าว่าง หากผู้ใช้ไม่กรอกข้อมูล ระบบจะแจ้งเตือนและให้กรอกใหม่

Input cannot be empty. Please try again.

ตรวจสอบทะเบียนรถซ้ำ

ก่อนเพิ่มรถเข้าสู่ **parkingStack** ระบบจะตรวจสอบว่า License Plate มีอยู่ใน Stack แล้วหรือไม่ เพื่อป้องกัน
ทะเบียนรถซ้ำ

ตรวจสอบตัวเลือก Menu

ระบบตรวจสอบตัวเลือกของผู้ใช้ให้อยู่ในช่วง **1-4** หากป้อนตัวเลือกอื่น ระบบจะแจ้งเตือนและให้เลือกใหม่

Error Handling

โปรแกรมมีการจัดการข้อผิดพลาดที่อาจเกิดขึ้นระหว่างการทำงาน เพื่อให้โปรแกรมสามารถทำงานต่อได้อย่างถูกต้อง

กรณี Stack ว่าง

ก่อนนำข้อมูลออกด้วย **pop()** ระบบจะตรวจสอบว่า Stack มีข้อมูลหรือไม่ หาก Stack ว่างจะไม่ดำเนินการ **pop()**

ผลลัพธ์:

Target Not Found

กรณีไม่พบรถเป้าหมาย

หากค้นหา License Plate แล้วไม่พบรถเป้าหมาย ระบบจะแสดง:

Target Not Found

และนำรถที่ถูกย้ายไปไว้ใน **temporaryStack** กลับคืนสู่ **parkingStack** เพื่อรักษาลำดับเดิม

กรณีข้อมูลไม่ถูกต้อง

หากผู้ใช้กรอกข้อมูลไม่ครบ หรือเลือก Menu ไม่ถูกต้อง ระบบจะแจ้งเตือนและให้ผู้ใช้ป้อนข้อมูลใหม่ แทนการทำให้โปรแกรมหยุด
ทำงาน

ขนาดข้อมูล

กำหนดขนาดข้อมูลที่ใช้ในการทดลองทั้งหมด 4 ระดับ ได้แก่

กำหนดขนาดข้อมูลที่ใช้ในการทดลองทั้งหมด 4 ระดับ ได้แก่

ลำดับ	ขนาดข้อมูล
-------	------------

1	100 รายการ
---	------------

2	1,000 รายการ
---	--------------

3	10,000 รายการ
---	---------------

4	50,000 รายการ
---	---------------

ข้อมูลแต่ละรายการแทนรถยนต์ 1 คัน โดยประกอบด้วย License Plate, Owner Name, Entry Time และ Vehicle Type ตามรูปแบบข้อมูลที่กำหนดไว้ในโครงงาน

การทดลองใช้ข้อมูลที่มี License Plate ไม่ซ้ำกัน และกำหนดรถเป้าหมายเพื่อใช้ในการค้นหา โดยพิจารณาทั้งกรณีที่รถเป้าหมายอยู่บริเวณด้านบนของ Stack และกรณีที่รถเป้าหมายอยู่ลึกลงไปใน Stack

จำนวนรอบทดลอง

เพื่อให้ผลการทดลองมีความน่าเชื่อถือมากขึ้น กำหนดให้ทดลองแต่ละขนาดข้อมูลอย่างน้อย 5 รอบ แล้วนำผลการทดลองมาคำนวณค่าเฉลี่ย

การวัดเวลาการทำงานใช้คำสั่ง **System.nanoTime()** ของภาษา Java โดยเริ่มจับเวลาก่อนเรียกใช้อัลกอริทึม และหยุดจับเวลาหลังจากอัลกอริทึมทำงานเสร็จ

ตัวอย่างการวัดเวลา

```
long start = System.nanoTime();
```

```
// เรียกใช้งาน Algorithm
```

```
long end = System.nanoTime();
```

```
long executionTime = end - start;
```

ค่าที่นำมาวิเคราะห์ประกอบด้วย

1. Execution Time
2. จำนวน Push
3. จำนวน Pop
4. จำนวน Comparison
5. จำนวน Cars Moved

วิธีการทดลอง

การทดลองดำเนินการตามขั้นตอนดังนี้

1. สร้างข้อมูลรถยนต์ตามขนาดที่กำหนด
2. เพิ่มข้อมูลรถยนต์เข้าสู่ **parkingStack**
3. กำหนด License Plate ของรถเป้าหมาย
4. เรียกใช้งาน Algorithm A: Iterative Method
5. บันทึกเวลาในการทำงานและจำนวน Operation
6. คืนค่า Stack ให้อยู่ในสภาพเริ่มต้น

7. เรียกใช้งาน Algorithm B: Recursive Method
8. บันทึกเวลาในการทำงานและจำนวน Operation
9. ทำการทดลองซ้ำอย่างน้อย 5 รอบ
10. คำนวณค่าเฉลี่ยของผลการทดลอง
11. นำผลของ Algorithm A และ Algorithm B มาเปรียบเทียบกัน

ตารางผลลัพธ์

ตารางที่ 7.1 ผลการทดลองด้านเวลา

ขนาดข้อมูล	Iterative (ns)	Recursive (ns)
100	[กรอกผลจริง]	[กรอกผลจริง]
1,000	[กรอกผลจริง]	[กรอกผลจริง]
10,000	[กรอกผลจริง]	[กรอกผลจริง]
50,000	[กรอกผลจริง]	[กรอกผลจริง]

ตารางที่ 7.2 ผลการทดลองด้านจำนวน Push

ขนาดข้อมูล	Iterative	Recursive
100	[กรอกผลจริง]	[กรอกผลจริง]
1,000	[กรอกผลจริง]	[กรอกผลจริง]
10,000	[กรอกผลจริง]	[กรอกผลจริง]

50,000	[กรอกผลจริง]	[กรอกผลจริง]
--------	--------------	--------------

ตารางที่ 7.3 ผลการทดลองด้านจำนวน Pop

ขนาดข้อมูล	Iterative	Recursive
100	[กรอกผลจริง]	[กรอกผลจริง]
1,000	[กรอกผลจริง]	[กรอกผลจริง]
10,000	[กรอกผลจริง]	[กรอกผลจริง]
50,000	[กรอกผลจริง]	[กรอกผลจริง]

ตารางที่ 7.4 ผลการทดลองด้านจำนวน Comparison

ขนาดข้อมูล	Iterative	Recursive
100	[กรอกผลจริง]	[กรอกผลจริง]
1,000	[กรอกผลจริง]	[กรอกผลจริง]
10,000	[กรอกผลจริง]	[กรอกผลจริง]
50,000	[กรอกผลจริง]	[กรอกผลจริง]

กราฟ

กราฟผลการทดลอง

จากผลการทดลองสามารถนำข้อมูลมาสร้างกราฟเพื่อเปรียบเทียบประสิทธิภาพของ Algorithm A และ Algorithm B โดยกราฟที่เหมาะสม ได้แก่

1. กราฟเปรียบเทียบขนาดข้อมูลกับ Execution Time
2. กราฟเปรียบเทียบขนาดข้อมูลกับจำนวน Push
3. กราฟเปรียบเทียบขนาดข้อมูลกับจำนวน Pop
4. กราฟเปรียบเทียบขนาดข้อมูลกับจำนวน Comparison

กราฟควรแสดงผลของ Iterative และ Recursive ในกราฟเดียวกัน เพื่อให้สามารถเปรียบเทียบการเปลี่ยนแปลงของประสิทธิภาพเมื่อขนาดข้อมูลเพิ่มขึ้นได้อย่างชัดเจน

การอภิปรายผล

จากการวิเคราะห์เชิงทฤษฎีในบทที่ 5 พบว่า Algorithm A และ Algorithm B มี Time Complexity เท่ากัน โดย Best Case มีค่าเป็น $O(1)$ ส่วน Average Case และ Worst Case มีค่าเป็น $O(n)$

เมื่อขนาดข้อมูลเพิ่มขึ้น จำนวนครั้งที่ต้องตรวจสอบมีแนวโน้มเพิ่มขึ้นตามจำนวนข้อมูล โดยเฉพาะในกรณีที่รถเป้าหมายอยู่บริเวณด้านล่างของ Stack หรือไม่พบรถเป้าหมาย ซึ่งเป็นกรณีที่ต้องตรวจสอบข้อมูลจำนวนมาก

สำหรับ Algorithm A จะใช้ **temporaryStack** เพื่อเก็บรถที่ถูกนำออกชั่วคราวระหว่างค้นหารถเป้าหมาย ขณะที่ Algorithm B ใช้ Recursive Call และ Call Stack ของระบบในการเก็บสถานะของการทำงาน

ดังนั้น แม้ทั้งสองวิธีจะมี Time Complexity ในเชิงทฤษฎีเท่ากัน แต่ลักษณะการใช้หน่วยความจำและรูปแบบการทำงานแตกต่างกัน โดย Algorithm A สามารถควบคุมการทำงานได้ง่ายกว่า และไม่มีความเสี่ยงต่อ Stack Overflow จาก Recursive Call

ผลการทดลองจริงควรนำมาเปรียบเทียบกับผลการวิเคราะห์เชิงทฤษฎี เพื่อพิจารณาว่าผลการทำงานของโปรแกรมมีแนวโน้มสอดคล้องกับ Complexity ที่วิเคราะห์ไว้หรือไม่

บทที่ 8 สรุป

8.1 สรุปผลการดำเนินงาน

โครงการนี้ได้ศึกษาและพัฒนาอัลกอริทึมสำหรับการค้นหาและนำรถยนต์เป้าหมายออกจากลานจอดรถที่มีทางเข้าและทางออกเพียงด้านเดียว โดยใช้โครงสร้างข้อมูล Stack ซึ่งทำงานตามหลัก LIFO (Last In First Out)

ในการดำเนินงานได้ออกแบบอัลกอริทึมจำนวน 2 วิธี ได้แก่ Algorithm A: Iterative Method และ Algorithm B: Recursive Method โดยทั้งสองวิธีสามารถค้นหาและนำรถยนต์เป้าหมายออกจาก Stack ได้ พร้อมทั้งนำรถที่ถูกย้ายออกชั่วคราวกลับเข้าสู่ Stack เพื่อรักษาลำดับของรถที่เหลืออยู่

นอกจากนี้ โปรแกรมได้รับการพัฒนาด้วยภาษา Java และมีการจัดการข้อมูลรถยนต์ การแสดงสถานะของ Stack การนับจำนวน Push, Pop และ Comparison รวมถึงการวัดเวลาในการทำงานของอัลกอริทึม

วิธีที่เหมาะสมกว่า

จากการวิเคราะห์และเปรียบเทียบอัลกอริทึมทั้งสองวิธี พบว่า Algorithm A: Iterative Method เหมาะสมกว่าสำหรับการนำไปใช้งานจริง

เนื่องจาก Iterative Method สามารถควบคุมการทำงานของ Stack ได้โดยตรง และใช้ **temporaryStack** สำหรับจัดเก็บข้อมูลที่ถูกย้ายชั่วคราว ทำให้สามารถติดตามลำดับการทำงานได้ง่าย

ในทางตรงกันข้าม Recursive Method ใช้ Recursive Call และ Call Stack ของระบบในการจัดเก็บสถานะการทำงาน หากมีข้อมูลจำนวนมาก อาจทำให้เกิดการใช้หน่วยความจำจาก Call Stack เพิ่มขึ้นและมีความเสี่ยงต่อ Stack Overflow

เหตุผลในการเลือก

เหตุผลที่เลือก Iterative Method เป็นวิธีที่เหมาะสมกว่าสำหรับระบบลानจอตรง มีดังนี้

1. สามารถควบคุมการทำงานได้ง่าย
2. โครงสร้างการทำงานเข้าใจง่าย
3. ไม่ต้องพึ่งพา Recursive Call
4. ไม่มีความเสี่ยงต่อ Stack Overflow จากการเรียกซ้ำจำนวนมาก
5. เหมาะกับการจัดการข้อมูลที่มีขนาดใหญ่
6. สามารถตรวจสอบและแก้ไขการทำงานของโปรแกรมได้ง่าย

จากการวิเคราะห์ในบทที่ 5 ยังพบว่าทั้ง Iterative และ Recursive มี Time Complexity เท่ากัน แต่ Iterative มีความเหมาะสมมากกว่าในกรณีที่ต้องนำไปใช้งานกับข้อมูลจำนวนมาก

ข้อจำกัด

โครงงานมีข้อจำกัดดังต่อไปนี้

1. โปรแกรมเป็นการจำลองระบบลานจอตรงโดยใช้ Stack และยังไม่ได้เชื่อมต่อกับระบบลานจอตรงจริง
2. ประสิทธิภาพด้านเวลาที่วัดได้อาจแตกต่างกันตามประสิทธิภาพของเครื่องคอมพิวเตอร์และสภาพแวดล้อมในการทดสอบ
3. Recursive Method มีข้อจำกัดด้าน Call Stack เมื่อจำนวนข้อมูลมีขนาดใหญ่
4. Iterative Method จำเป็นต้องใช้ **temporaryStack** เพิ่มเติมสำหรับเก็บข้อมูลที่ถูกรวบรวม
5. การทดลองต้องใช้ผลจากการรันโปรแกรมจริงเพื่อให้สามารถเปรียบเทียบเวลาและจำนวน Operation ได้อย่างถูกต้อง

แนวทางพัฒนาต่อ

สามารถพัฒนาโครงการต่อได้ดังนี้

1. พัฒนาระบบให้มี Graphical User Interface (GUI) เพื่อให้ผู้ใช้สามารถจัดการรถยนต์ผ่านหน้าจอได้ง่ายขึ้น
2. เพิ่มระบบฐานข้อมูลสำหรับจัดเก็บข้อมูลรถยนต์และประวัติการเข้า-ออก
3. เพิ่มระบบค้นหารถยนต์จากข้อมูลอื่น เช่น ชื่อเจ้าของหรือประเภทรถ
4. เพิ่มระบบบันทึกประวัติการนำรถเข้าและออกจากลานจอด
5. รองรับลานจอดรถที่มีหลายช่องทางเข้าและออก
6. เพิ่มระบบสร้างกราฟผลการทดลองโดยอัตโนมัติ
7. เพิ่มการทดสอบกับข้อมูลขนาดใหญ่ขึ้น เพื่อศึกษาข้อจำกัดของแต่ละอัลกอริทึม

สิ่งที่ได้เรียนรู้

จากการจัดทำโครงการ สมาชิกได้เรียนรู้เกี่ยวกับการวิเคราะห์ปัญหาและการเลือกโครงสร้างข้อมูลที่เหมาะสม โดยเฉพาะการนำ Stack ซึ่งทำงานตามหลัก LIFO มาประยุกต์ใช้กับปัญหาการจัดการลานจอดรถ

นอกจากนี้ยังได้เรียนรู้การออกแบบอัลกอริทึมทั้งแบบ Iterative และ Recursive การวิเคราะห์ Time Complexity และ Space Complexity การพิสูจน์ความถูกต้องของอัลกอริทึม การพัฒนาโปรแกรมด้วยภาษา Java รวมถึงการวัดและเปรียบเทียบประสิทธิภาพจากการทดลองจริง

การทำโครงการยังช่วยฝึกทักษะการทำงานเป็นทีม การแบ่งหน้าที่ การแก้ไขปัญหา และการจัดทำรายงานทางวิชาการอย่างเป็นระบบ ซึ่งสอดคล้องกับวัตถุประสงค์ของโครงการที่กำหนดไว้

